The consequence of what we (informally) proved[18] above is that, instead of comparing each point in a cell to all the points close to the border of the adjacent cells, we can just keep track of the core points within a distance ε from a cell's border (or 2ε from a shard's border), and merge those clusters that have a point in either inner or outer margin that is a core point in either of the adjacent cells.

Figure 13.13 shows an example of how a MapReduce job would perform a distributed clustering of a 2D dataset using DBSCAN and the reduction described above.

The first step is sharding the dataset according to a regular grid: each grid cell is then expanded along all directions with a further area of side ε, and shards are formed by filtering all points within these expanded rectangles (which, it's important to remind, overlaps along their common edges with adjacent cells).

Each mapper performs DBSCAN clustering on its shard, and at the next step then a reducer is spun up for each pair of adjacent shards (in this example we used a 2x2 grid, so there are 4 pairs of adjacent grid cells).

Listing 13.5 MapReduce DBSCAN

```
function MRdbscan(points, numShards, eps, minPts)
                              #1
  shards, adjList, marginPoints ← gridShard(points, numShards,
eps)           #2
  mappers ← initMappers(numShards, dbscan, shards, eps,
minPts)               #3
  reducers ← initReducers(adjList, mergeClusters,
marginPoints)               #4
```